
Software Requirements Specification

for

Using federated learning to detect intrusions in IOT devices with ensuring efficient communication

Version 1.0 approved

**Prepared by
Umang Godnaha
Tanushree Gadabay
Aditya Vardhan
M. Tejesh Goud
Y. Sudeep Reddy**

11/08/26

Table of Contents

Table of Contents	ii
Revision History	ii
1. Introduction	1
1.1 Purpose	1
1.2 Document Conventions	2
1.3 Intended Audience and Reading Suggestions	2
1.4 Product Scope	2
1.5 References	3
2. Overall Description	5
2.1 Product Perspective	5
2.2 Product Functions	5
2.3 User Classes and Characteristics	6
2.4 Operating Environment	7
2.5 Design and Implementation Constraints	8
2.6 User Documentation	8
2.7 Assumptions and Dependencies	9
3. External Interface Requirements	9
3.1 User Interfaces	9
3.2 Hardware Interfaces	10
3.3 Software Interfaces	10
3.4 Communications Interfaces	11
4. System Features	12
4.1 Data Preprocessing Module	12
4.2 Local TCN Training Module	13
4.3 Federated Learning and Model Aggregation	13
4.4 Intrusion Detection and Attack Classification	14
4.5 Performance Evaluation and Report Generation	14
5. Other Nonfunctional Requirements	15
5.1 Performance Requirements	15
5.2 Safety Requirements	15
5.3 Security Requirements	15
5.4 Software Quality Attributes	15
5.5 Business Rules	16
5.6 Design Constraints	16
5.7 Assumptions and Dependencies	16
6. UML Diagrams	17
6.1 Class Diagram	18
6.2 Sequence Diagram	18
6.3 Use Case Diagram	19
6.4 Activity Diagram	20
6.5 Component Diagram	21
6.6 Deployment Diagram	22
Appendix A: Glossary	23
Appendix B: Analysis Models	24
Appendix C: To Be Determined List	25
Appendix D: Conclusion	25
Bibliography	25

Revision History

Name	Date	Reason For Changes	Version

--	--	--	--

1. Introduction

The Internet of Things (IoT) has transformed the way devices communicate and exchange information by connecting billions of smart devices through the internet. These devices are widely used in smart homes, healthcare, industrial automation, agriculture, transportation, and many other domains. Although IoT technology offers numerous advantages such as automation, real-time monitoring, and remote control, it also introduces several security challenges. Due to their limited computational power, memory, and battery capacity, most IoT devices cannot support complex security mechanisms, making them vulnerable to various cyberattacks including Denial of Service (DoS), Distributed Denial of Service (DDoS), brute-force attacks, malware, botnets, and data injection attacks.

Traditional Intrusion Detection Systems (IDS) generally rely on centralized machine learning models where network traffic from all devices is collected and processed at a central server. While this approach provides good detection accuracy, it raises privacy concerns, increases communication overhead, and creates a single point of failure. These limitations make centralized approaches unsuitable for large-scale IoT environments.

To overcome these challenges, this project proposes a privacy-preserving Intrusion Detection System using Federated Learning (FL) and a Temporal Convolutional Network (TCN). Instead of sending raw network data to a central server, each IoT device trains the TCN model locally using its own data. Only the updated model parameters are shared with the federated server, where they are aggregated using the Federated Averaging (FedAvg) algorithm to generate a global model. This process significantly reduces communication costs while preserving user privacy.

The Temporal Convolutional Network is selected because it efficiently captures temporal patterns present in sequential network traffic while requiring lower computational resources compared to recurrent neural networks. This makes it an ideal choice for resource-constrained IoT devices.

The proposed system aims to provide a lightweight, scalable, and accurate intrusion detection framework capable of identifying malicious network activities with minimal communication overhead while maintaining data privacy. The system will be evaluated using publicly available IoT intrusion detection datasets and standard evaluation metrics such as Accuracy, Precision, Recall, F1-Score, and ROC-AUC.

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to define the functional and non-functional requirements of the proposed Intrusion Detection System based on Temporal Convolutional Networks (TCN) and Federated Learning for resource-constrained IoT devices. This document serves as a complete reference for understanding the objectives, architecture, system functionalities, performance expectations, and implementation requirements of the proposed software.

The SRS provides a clear understanding of the system requirements for developers, project guides, testers, and future users. It acts as a communication medium between stakeholders by clearly describing how the proposed system should operate and what functionalities it should provide. The document also serves as a foundation for system design, implementation, testing, deployment, and future maintenance.

The primary objective of the proposed system is to develop a lightweight intrusion detection model that can accurately detect malicious network traffic while preserving user privacy through Federated Learning. By training the model locally on IoT devices and sharing only model parameters instead of raw data, the system reduces communication overhead and protects sensitive information. The TCN model is utilized to efficiently learn temporal dependencies in network traffic, making the proposed solution suitable for deployment in resource-limited IoT environments.

1.2 Document Conventions

This Software Requirements Specification (SRS) follows the standard IEEE SRS structure to ensure consistency and clarity throughout the document. The document is organized into different sections that describe the overall system, its functional and non-functional requirements, interfaces, system features, and additional requirements.

The following conventions are used throughout this document:

- **Headings** represent the major sections of the SRS.
- **Subheadings** divide each section into smaller topics for better understanding.
- Important technical terms such as **Temporal Convolutional Network (TCN)**, **Federated Learning (FL)**, **Internet of Things (IoT)**, and **Intrusion Detection System (IDS)** are written using their standard abbreviations after their first occurrence.
- Functional requirements are uniquely identified using requirement IDs (e.g., FR-01, FR-02) to simplify implementation and testing.
- The words **shall**, **should**, and **may** are used according to their intended meaning. "Shall" indicates mandatory requirements, "should" indicates recommended requirements, and "may" indicates optional features.

All diagrams, tables, and figures included in this document are numbered and titled appropriately for easy reference.

1.3 Intended Audience and Reading Suggestions

This document is intended for all stakeholders involved in the development and evaluation of the proposed intrusion detection system. It provides a detailed description of the system requirements and serves as a common reference throughout the software development life cycle.

The primary audience includes project developers, who will use this document to understand the system requirements and implement the proposed solution. Project guides and faculty members can use it to evaluate whether the project satisfies its intended objectives. Testers can refer to the functional and non-functional requirements while preparing test cases to validate the system. Researchers and future developers may also use this document as a reference for extending or improving the proposed system.

Readers are encouraged to begin with the Introduction to understand the project objectives, followed by the Overall Description to gain an understanding of the system architecture and functionality. The External Interface Requirements and System Features sections provide detailed information regarding system behavior, while the Non-Functional Requirements section describes the expected quality, security, and performance characteristics of the software.

1.4 Product Scope

The proposed project focuses on developing a lightweight and privacy-preserving Intrusion Detection System for resource-constrained Internet of Things (IoT) devices using Temporal Convolutional Networks (TCN) and Federated Learning. The system is designed to identify malicious network traffic while minimizing computational overhead and protecting sensitive user data.

Unlike traditional centralized intrusion detection systems that require all network traffic to be transmitted to a central server, the proposed system performs model training locally on each IoT

device. Only the trained model parameters are shared with the federated server, where they are aggregated using the Federated Averaging (FedAvg) algorithm to generate an improved global model. This approach significantly reduces communication costs and preserves user privacy.

The Temporal Convolutional Network is employed because of its ability to effectively learn temporal dependencies from sequential network traffic while maintaining low computational complexity. The proposed system supports preprocessing of IoT network traffic datasets, local model training, federated model aggregation, intrusion detection, attack classification, and performance evaluation.

The main objectives of the proposed software are:

- To accurately detect malicious network traffic in IoT environments.
- To preserve user privacy by avoiding the transmission of raw data.
- To reduce communication overhead through Federated Learning.
- To develop a lightweight model suitable for resource-constrained IoT devices.
- To improve the overall security and reliability of IoT networks.

The system is intended for research and educational purposes and can be further extended for deployment in real-world smart homes, healthcare systems, industrial IoT environments, and smart city applications.

1.5 References

The proposed system has been designed and developed by referring to research papers, software documentation, and standard development guidelines related to Federated Learning, Temporal Convolutional Networks, and Intrusion Detection Systems. The following references were used during the analysis, design, and implementation of this project.

1. Primary Research Paper

S. Anbazhagan, M. Kavitha, S. Murugan, K. M. Kumar, M. M. Rashid, and A. M. Hilal, "**Federated Learning-Based Lightweight Intrusion Detection for Resource-Constrained IoT Devices Using Temporal Convolutional Networks**," Scientific Reports, Nature Portfolio, 2025.

2. IEEE Recommended Practice for Software Requirements Specifications (IEEE SRS Standard).

3. TensorFlow Documentation. <https://www.tensorflow.org/>

4. Keras Documentation. <https://keras.io/>

5. Scikit-learn Documentation. <https://scikit-learn.org/>

6. NumPy Documentation. <https://numpy.org/>

7. **Pandas** **Documentation.**
<https://pandas.pydata.org/>
8. **Research papers and documentation related to Federated Learning, Temporal Convolutional Networks (TCN), Internet of Things (IoT), and Intrusion Detection Systems (IDS)** that were referred to during the study and implementation of the proposed system.

2. Overall Description

*The proposed system is a privacy-preserving Intrusion Detection System (IDS) designed specifically for resource-constrained Internet of Things (IoT) devices. It combines the capabilities of **Federated Learning (FL)** and **Temporal Convolutional Networks (TCN)** to detect malicious network activities while maintaining data privacy and reducing communication overhead. Unlike traditional centralized IDS solutions, the proposed system performs model training locally on IoT devices, allowing sensitive data to remain on the device itself.*

Each participating IoT device independently trains a local TCN model using its own network traffic data. Instead of transmitting raw data to a central server, only the updated model parameters are shared. These parameters are aggregated at the federated server using the Federated Averaging (FedAvg) algorithm to generate a global model. The updated global model is then distributed back to all participating devices, enabling collaborative learning without compromising user privacy.

The system is designed to identify various types of network attacks while maintaining high detection accuracy and low computational overhead. The lightweight nature of the TCN architecture makes the proposed solution suitable for IoT environments where devices have limited processing power, memory, and energy resources. The system can be trained and evaluated using publicly available IoT intrusion detection datasets and can be extended to support real-world smart environments in the future.

2.1 Product Perspective

The proposed software is a standalone machine learning-based intrusion detection system developed for IoT networks. It acts as a security layer that continuously analyzes network traffic and identifies malicious activities before they can compromise the network. The system is intended to work in a federated environment where multiple IoT devices collaboratively train a common intrusion detection model without sharing their local datasets.

The overall architecture consists of three major components:

- **IoT Client Devices:** *These devices collect network traffic, preprocess the data, and locally train the TCN model using their own datasets.*
- **Federated Server:** *The central server receives only the locally trained model parameters from each client, aggregates them using the Federated Averaging (FedAvg) algorithm, and generates a new global model.*
- **Global Intrusion Detection Model:** *The updated global model is distributed back to all participating clients, allowing every device to benefit from collective learning while maintaining complete data privacy.*

The proposed architecture eliminates the need for centralized data collection, thereby reducing bandwidth consumption, communication costs, and privacy risks.

2.2 Product Functions

The proposed intrusion detection system provides several important functions to ensure efficient and secure detection of cyberattacks in IoT networks. The major functions of the software are listed

below:

- *Import and preprocess IoT network traffic datasets.*
- *Handle missing values, encode categorical features, and normalize the dataset.*
- *Divide the dataset among multiple federated clients for local training.*
- *Train a Temporal Convolutional Network (TCN) independently on each client.*
- *Share only trained model parameters with the federated server.*
- *Aggregate local models using the Federated Averaging (FedAvg) algorithm.*
- *Generate an improved global intrusion detection model after each communication round.*
- *Distribute the updated global model back to all participating clients.*
- *Detect and classify normal and malicious network traffic.*
- *Evaluate model performance using Accuracy, Precision, Recall, F1-Score, and ROC-AUC.*
- *Reduce communication overhead while preserving data privacy.*
- *Generate performance reports and visualization graphs for analysis.*

The system is designed to operate efficiently even on resource-constrained IoT devices without significantly affecting their normal operation.

2.3 User Classes and Characteristics

The proposed system is designed for users who are involved in monitoring, managing, or researching the security of IoT networks. Since the project is developed as a machine learning-based intrusion detection system, different categories of users may interact with it based on their roles and responsibilities.

1. Network Administrator

The network administrator is responsible for monitoring the IoT network and analyzing intrusion detection results. This user can load network traffic data, view attack predictions, monitor model performance, and generate reports. The administrator should have a basic understanding of networking concepts and cybersecurity.

2. Researcher or Data Scientist

Researchers use the system to train and evaluate the Temporal Convolutional Network (TCN) model using different IoT datasets. They may modify training parameters, evaluate the effectiveness of Federated Learning, compare performance metrics, and analyze the model's accuracy. This user should have knowledge of machine learning, Python programming, and deep learning concepts.

3. System Developer

The developer is responsible for implementing, testing, and maintaining the software. This user

manages the preprocessing pipeline, TCN model architecture, Federated Learning workflow, and system integration. The developer should be familiar with Python, TensorFlow/Keras, Federated Learning concepts, and software development practices.

4. Faculty or Project Evaluator

Faculty members or project evaluators use the software to understand the working of the proposed system and evaluate its performance. They mainly analyze the system architecture, experimental results, and intrusion detection accuracy rather than interacting with the training process.

2.4 Operating Environment

The proposed intrusion detection system is developed using Python and deep learning libraries. It is designed to run on Windows, Linux, or cloud-based platforms such as Google Colab and Kaggle. The system requires a Python environment with the necessary machine learning libraries installed.

The software can be executed on a standard personal computer for development and testing purposes. If available, a GPU can be used to accelerate model training; however, the system is also capable of running on CPU-only systems. The federated learning process is simulated by dividing the dataset among multiple virtual clients during experimentation.

Software Environment

- *Operating System: Windows 10/11 or Linux*
- *Programming Language: Python 3.10 or above*
- *Development Environment: Jupyter Notebook, Google Colab, VS Code, or Kaggle*
- *Deep Learning Framework: TensorFlow/Keras*
- *Libraries: NumPy, Pandas, Scikit-learn, Matplotlib, TensorFlow*
- *Federated Learning Framework: Custom FedAvg implementation or Flower (if used)*

Hardware Environment

- *Processor: Intel Core i5 or equivalent (minimum)*
- *RAM: 8 GB minimum (16 GB recommended)*
- *Storage: Minimum 10 GB free space*
- *Internet Connection: Required for downloading datasets and software packages*
- *GPU: NVIDIA GPU (optional but recommended for faster training)*

2.5 Design and Implementation Constraints

The development of the proposed system is subject to several design and implementation

constraints.

Since the project targets resource-constrained IoT environments, the model must remain lightweight while maintaining high intrusion detection accuracy. Excessive computational complexity may reduce its suitability for IoT devices.

The availability and quality of datasets also influence the overall performance of the model. The proposed system is trained using publicly available IoT intrusion detection datasets, and the performance may vary depending on the dataset characteristics and class distribution.

The Federated Learning process assumes that multiple clients participate in local model training and communicate only model parameters with the central server. Communication delays, network instability, or limited client participation may affect the convergence of the global model.

The implementation depends on Python libraries such as TensorFlow, NumPy, Pandas, and Scikit-learn. Any compatibility issues between library versions may require additional configuration before execution.

Hardware limitations such as limited RAM or the absence of GPU acceleration may increase the model training time, especially when working with large IoT datasets.

2.6 User Documentation

The proposed software will include basic user documentation to help users understand and operate the system effectively. The documentation will contain installation instructions, software requirements, dataset preparation guidelines, and steps for executing the intrusion detection model.

The user guide will explain how to load datasets, preprocess network traffic, initiate local model training, perform federated aggregation, evaluate model performance, and interpret the generated results. It will also include descriptions of important performance metrics such as Accuracy, Precision, Recall, F1-Score, and ROC-AUC.

In addition to the user guide, comments will be included within the source code to improve readability and simplify future maintenance or enhancements

2.7 Assumptions and Dependencies

<List any assumed factors (as opposed to known facts) that could affect the requirements stated in the SRS. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project, unless they are already documented elsewhere (for example, in the vision and scope document or the project plan).>

3. External Interface Requirements

The External Interface Requirements describe how the proposed intrusion detection system interacts with users, hardware components, software modules, and communication networks. These interfaces ensure smooth operation of the system and facilitate data exchange between different components. Since the proposed system is implemented using Federated Learning, communication between IoT clients and the central server is an essential part of the software architecture.

3.1 User Interfaces

The proposed system provides a simple and user-friendly interface that allows users to perform intrusion detection and evaluate the performance of the model. The interface is primarily intended for researchers, developers, and network administrators.

The user interface allows users to:

- *Load the IoT intrusion detection dataset.*
- *Start the data preprocessing process.*
- *Configure model parameters such as batch size, learning rate, number of communication rounds, and number of local epochs.*
- *Initiate local training on federated clients.*
- *Execute the Federated Learning process.*
- *View the training progress of each communication round.*
- *Display the aggregated global model performance.*
- *Predict whether network traffic is normal or malicious.*
- *View performance metrics including Accuracy, Precision, Recall, F1-Score, and ROC-AUC.*
- *Display graphs such as training loss, validation accuracy, confusion matrix, and ROC curve.*
- *Save the trained model and generated evaluation reports.*

The interface is designed to be simple so that users with basic knowledge of machine learning can operate the software without difficulty.

3.2 Hardware Interfaces

The proposed system does not require any specialized hardware and can be executed on a standard desktop or laptop computer. During experimentation, multiple IoT devices are simulated as federated clients. In real-world deployment, these clients may represent smart sensors, smart cameras, wearable devices, industrial controllers, or other IoT devices connected to the network.

The hardware components involved in the system include:

- *IoT devices that collect and process local network traffic.*
- *A central server responsible for aggregating locally trained model parameters using the Federated Averaging (FedAvg) algorithm.*
- *Personal computer or workstation used for model development, testing, and performance evaluation.*
- *Optional GPU for accelerating deep learning model training.*
- *Stable internet or local network connection for communication between federated clients and the central server.*

The system has been designed to minimize computational requirements, making it suitable for deployment on resource-constrained IoT devices.

3.3 Software Interfaces

The proposed software is developed using Python and integrates several open-source libraries and frameworks for data preprocessing, deep learning, and model evaluation.

The software interfaces include:

Software	Purpose
Python	Programming language used for implementation
TensorFlow / Keras	Development and training of the Temporal Convolutional Network
NumPy	Numerical computations
Pandas	Data preprocessing and manipulation
Scikit-learn	Data preprocessing and performance evaluation
Matplotlib	Visualization of training results and evaluation graphs
Google Colab / Kaggle / VS Code	Development and execution environment

These software components work together to perform data preprocessing, model training, federated aggregation, intrusion detection, and performance evaluation.

3.4 Communications Interfaces

Communication plays an important role in the proposed Federated Learning architecture. Unlike traditional centralized systems, the proposed software does not transmit raw network traffic from IoT devices to the server. Instead, each participating client performs local model training using its own dataset and shares only the trained model parameters with the federated server.

The communication process consists of the following steps:

- 1. The federated server initializes the global TCN model and distributes it to all participating IoT clients.*
- 2. Each client performs local training using its private dataset.*
- 3. After completing local training, the clients send only the updated model parameters to the central server.*
- 4. The central server aggregates the received parameters using the Federated Averaging (FedAvg) algorithm.*
- 5. The updated global model is distributed back to all participating clients.*
- 6. This process continues for multiple communication rounds until the model converges.*

Since only model parameters are exchanged instead of raw data, the communication overhead is significantly reduced while ensuring better privacy and security of sensitive information. The communication interface supports secure transmission of model updates between clients and the central server over standard network protocols.

4. System Features

The proposed intrusion detection system consists of multiple functional modules that work together to detect malicious activities in IoT networks while preserving data privacy. Each module performs a specific task, starting from data preprocessing to federated model aggregation and attack prediction. The following sections describe the major features of the proposed software.

4.1 Data Preprocessing Module

Description and Priority

The Data Preprocessing Module is responsible for preparing the raw IoT network traffic dataset before it is used for model training. Since real-world datasets often contain missing values, duplicate records, inconsistent data formats, and categorical features, preprocessing is necessary to improve the quality of the data and enhance the performance of the intrusion detection model.

This module performs operations such as data cleaning, handling missing values, encoding categorical features, normalization, feature selection, and dataset partitioning. Once preprocessing is completed, the dataset is divided into multiple client datasets to simulate the Federated Learning environment.

Priority: High

Stimulus / Response Sequences

Stimulus

- *User uploads the IoT intrusion detection dataset.*
- *User initiates the preprocessing process.*

Response

- *The system validates the dataset.*
- *Missing or duplicate values are handled.*
- *Categorical attributes are converted into numerical values.*
- *Features are normalized.*
- *Dataset is partitioned among federated clients.*
- *The processed dataset is prepared for model training.*

Functional Requirements

FR-01: *The system shall allow the user to upload an IoT intrusion detection dataset.*

FR-02: The system shall validate the uploaded dataset before processing.

FR-03: The system shall remove duplicate and inconsistent records.

FR-04: The system shall encode categorical features into numerical values.

FR-05: The system shall normalize numerical features before training.

FR-06: The system shall partition the dataset into multiple client datasets for Federated Learning.

4.2 Local TCN Training Module

Description and Priority

This module is responsible for training the Temporal Convolutional Network (TCN) on each federated client. Every client independently trains the model using its local dataset without transmitting raw data to the central server. This ensures that sensitive information remains within the client device while enabling collaborative model learning.

The TCN architecture captures temporal dependencies in sequential network traffic and produces an efficient intrusion detection model suitable for resource-constrained IoT devices.

Priority: High

Stimulus / Response Sequences

Stimulus

- The federated server distributes the initial global model.
- The client begins local model training.

Response

- Local data is loaded.
 - The TCN model is trained using the local dataset.
 - Model weights are updated.
 - The trained model parameters are prepared for transmission to the federated server.
-

Functional Requirements

FR-07: The system shall initialize the Temporal Convolutional Network on every client.

FR-08: The system shall train the model using only locally available data.

FR-09: The system shall prevent raw dataset transmission during training.

FR-10: The system shall generate updated model parameters after local training.

4.3 Federated Learning and Model Aggregation

Description and Priority

The Federated Learning module coordinates collaborative model training among multiple IoT clients. Instead of collecting user data, the central server receives only the trained model parameters from participating clients. These parameters are aggregated using the Federated Averaging (FedAvg) algorithm to generate an updated global model.

The global model is then redistributed to all clients for the next communication round until satisfactory performance is achieved.

Priority: High

Stimulus / Response Sequences

Stimulus

- Local clients complete model training.
- Model parameters are transmitted to the federated server.

Response

- The server receives model parameters.
- FedAvg aggregation is performed.
- A new global model is generated.
- The updated model is distributed back to all clients.

Functional Requirements

FR-11: The system shall receive model parameters from all participating clients.

FR-12: The system shall aggregate local model parameters using the Federated Averaging (FedAvg) algorithm.

FR-13: The system shall generate an updated global model after each communication round.

FR-14: The system shall distribute the updated global model to all participating clients.

FR-15: The system shall repeat the aggregation process until the predefined number of communication rounds is completed.

4.4 Intrusion Detection and Attack Classification

Description and Priority

After the global model has been trained, the system uses the trained TCN model to classify incoming network traffic as either normal or malicious. The model analyzes sequential traffic patterns and identifies suspicious activities that may indicate cyberattacks.

The system supports the detection of multiple attack categories depending on the selected dataset.

Priority: High

Stimulus / Response Sequences**Stimulus**

- New network traffic is received.

Response

- Data is preprocessed.
- The trained TCN model performs prediction.
- The traffic is classified as normal or malicious.
- Prediction results are displayed to the user.

Functional Requirements

FR-16: The system shall classify incoming traffic using the trained TCN model.

FR-17: The system shall identify malicious traffic patterns.

FR-18: The system shall display prediction results to the user.

FR-19: The system shall support classification of multiple attack categories when supported by the dataset.

4.5 Performance Evaluation and Report Generation

Description and Priority

This module evaluates the effectiveness of the intrusion detection model using standard machine learning performance metrics. It also generates graphical representations and summary reports that help users analyze the model's performance after training.

Priority: Medium

Stimulus / Response Sequences**Stimulus**

- Model training is completed.

Response

- The model is evaluated on the testing dataset.
- Performance metrics are calculated.
- Graphs and reports are generated.

Functional Requirements

FR-20: The system shall calculate Accuracy, Precision, Recall, F1-Score, and ROC-AUC.

FR-21: The system shall generate a confusion matrix for model evaluation.

FR-22: The system shall display training and validation performance graphs.

FR-23: The system shall allow users to save evaluation results for future analysis.

5. Other Nonfunctional Requirements

Non-functional requirements describe the quality attributes and constraints that the proposed intrusion detection system must satisfy. Unlike functional requirements, these requirements focus on system performance, reliability, security, maintainability, and overall usability. These characteristics ensure that the software operates efficiently in resource-constrained IoT environments while providing accurate and reliable intrusion detection.

5.1 Performance Requirements

The proposed intrusion detection system should provide high detection accuracy while maintaining low computational and communication overhead. Since the system is intended for resource-constrained IoT devices, it should efficiently utilize available system resources without affecting the normal operation of the devices.

The TCN model should process network traffic within a reasonable amount of time and produce predictions with minimal latency. During federated training, only model parameters should be transmitted between the clients and the central server to reduce network bandwidth consumption.

The system should achieve satisfactory intrusion detection performance using standard evaluation metrics such as Accuracy, Precision, Recall, F1-Score, and ROC-AUC. The software should also support multiple communication rounds without significantly increasing execution time.

5.2 Safety Requirements

The proposed system does not directly control any physical hardware; therefore, it does not introduce risks related to physical safety. However, incorrect intrusion detection may affect the reliability of network security monitoring.

To minimize such risks, the software should validate input data before processing and generate appropriate error messages whenever invalid or corrupted datasets are detected. The system should prevent unexpected termination during model training and maintain data consistency throughout the federated learning process. Appropriate exception handling mechanisms should be implemented to ensure that software failures do not interrupt the overall execution of the intrusion detection system.

5.3 Security Requirements

Security is one of the primary objectives of the proposed system. The software is designed to preserve user privacy by ensuring that raw network traffic data remains on the local IoT devices throughout the training process.

Only trained model parameters are transmitted to the federated server for aggregation. This significantly reduces the possibility of exposing sensitive user information compared to centralized machine learning approaches.

The system should ensure secure communication between IoT clients and the federated server during the exchange of model updates. Access to the software should be limited to authorized users, and stored

datasets should be protected from unauthorized modifications.

The software should also maintain the confidentiality, integrity, and availability of both the trained models and evaluation results.

5.4 Software Quality Attributes

The proposed software is expected to satisfy the following quality attributes.

Reliability

The system should consistently detect malicious network traffic with high accuracy under different operating conditions. It should continue functioning correctly even when individual clients temporarily disconnect during federated training.

Availability

The intrusion detection system should remain available whenever network traffic needs to be monitored. The software should recover gracefully from temporary failures and resume execution without affecting previously completed training rounds.

Maintainability

The software should be developed using a modular architecture so that individual components such as data preprocessing, TCN model training, federated aggregation, and evaluation can be modified or upgraded independently without affecting the remaining modules.

Scalability

The proposed system should support an increasing number of IoT devices participating in the Federated Learning process. Additional clients should be incorporated without requiring major architectural modifications.

Usability

The software should provide a simple and easy-to-understand interface that enables users to execute model training, perform intrusion detection, and evaluate results with minimal technical complexity.

Portability

The software should be executable on multiple operating systems including Windows and Linux. It should also support cloud-based development environments such as Google Colab and Kaggle with minimal configuration changes.

Efficiency

The Temporal Convolutional Network should utilize system resources efficiently while maintaining high intrusion detection performance. The federated learning process should reduce communication overhead by exchanging only model parameters instead of raw datasets.

Accuracy

The system should achieve high intrusion detection accuracy and minimize false positive and false negative rates. Performance should be evaluated using commonly accepted machine learning metrics including Accuracy, Precision, Recall, F1-Score, and ROC-AUC.

5.5 Business Rules

The proposed system is intended for educational and research purposes. The software shall be used to study the effectiveness of Federated Learning and Temporal Convolutional Networks for intrusion detection in resource-constrained IoT environments.

The system shall comply with data privacy principles by ensuring that local datasets are not transmitted to the central server during the federated training process. Any future deployment in real-world environments should comply with the security policies and regulations applicable to the organization using the software.

5.6 Design Constraints

The development of the proposed system is subject to the following constraints:

- The software is implemented using Python and TensorFlow/Keras.*
- The Temporal Convolutional Network architecture is used as the primary deep learning model.*
- Federated Learning is implemented using the Federated Averaging (FedAvg) algorithm.*
- Publicly available IoT intrusion detection datasets are used for training and evaluation.*
- The software is designed to operate efficiently on systems with limited computational resources.*
- The performance of the system depends on the quality and distribution of the training dataset.*

- The number of federated clients used during experimentation may be limited by the available hardware resources.

5.7 Assumptions and Dependencies

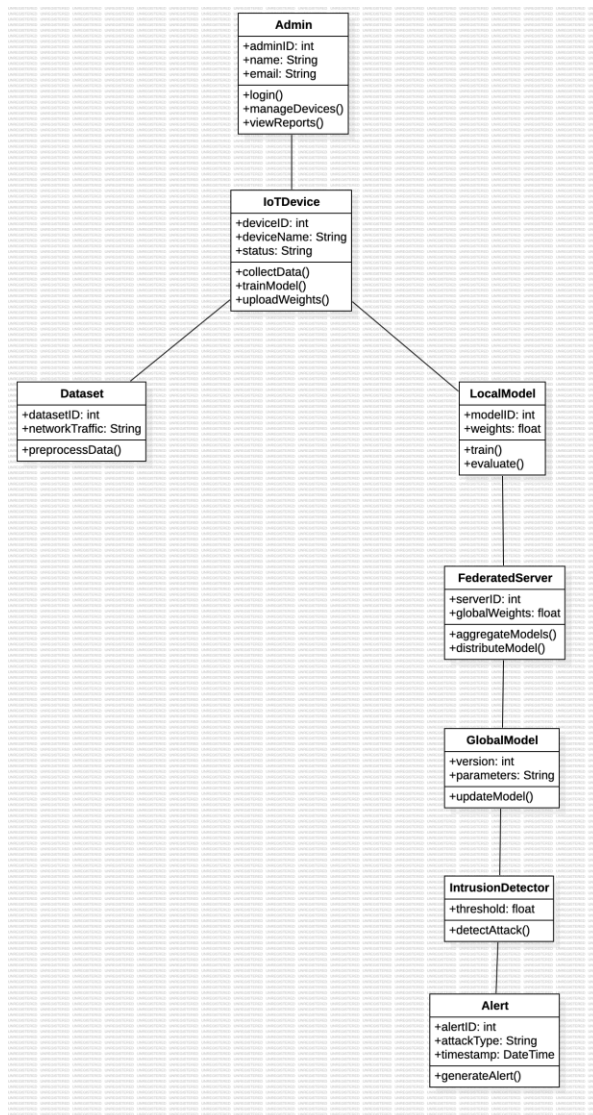
The proposed software assumes that each participating client has sufficient computational capability to perform local model training. It also assumes that a reliable communication channel exists between the clients and the federated server for exchanging model parameters.

The system depends on the successful installation of required Python libraries such as TensorFlow, Keras, NumPy, Pandas, Scikit-learn, and Matplotlib. It also assumes that the selected IoT dataset is correctly formatted and contains labeled network traffic samples required for supervised learning.

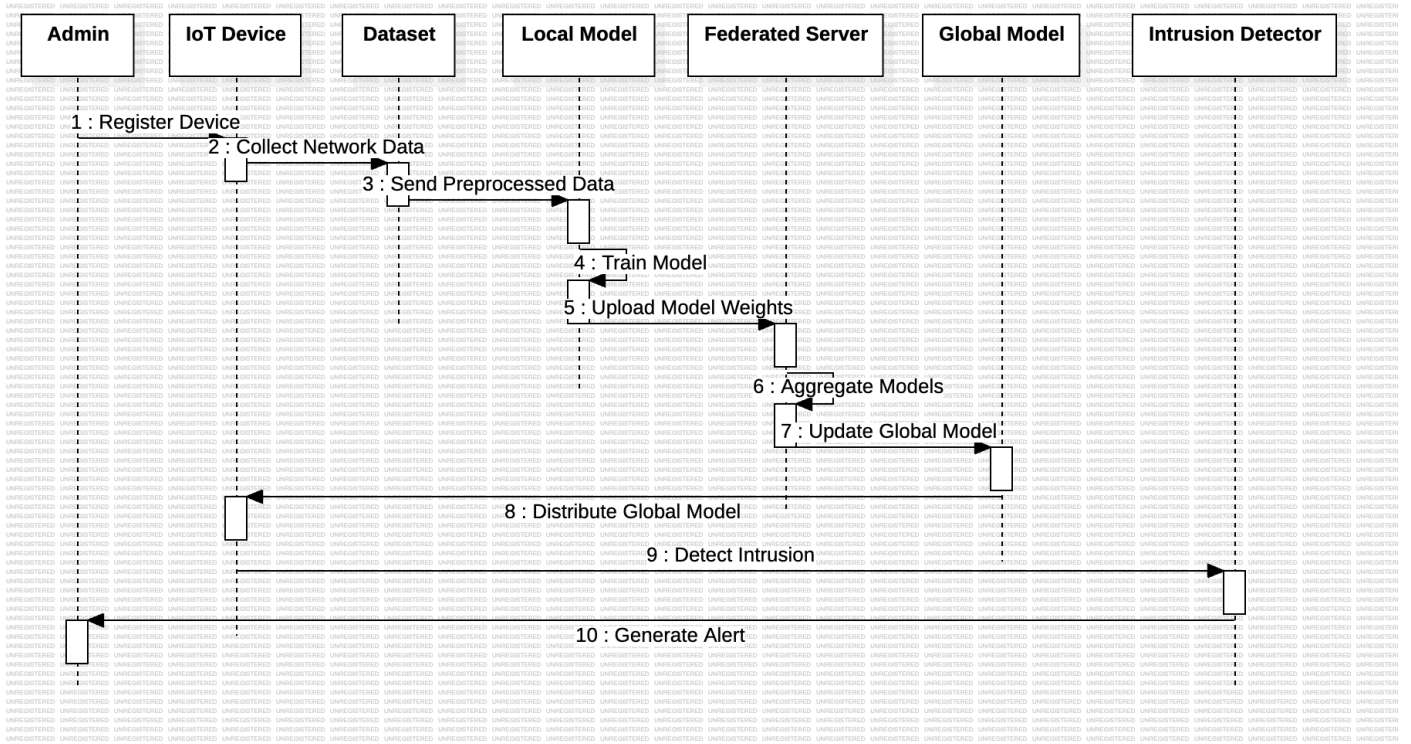
The effectiveness of the intrusion detection model depends on the quality, diversity, and representativeness of the training data. Future improvements in the dataset or learning algorithms may further enhance the system's performance.

6. UML Diagrams

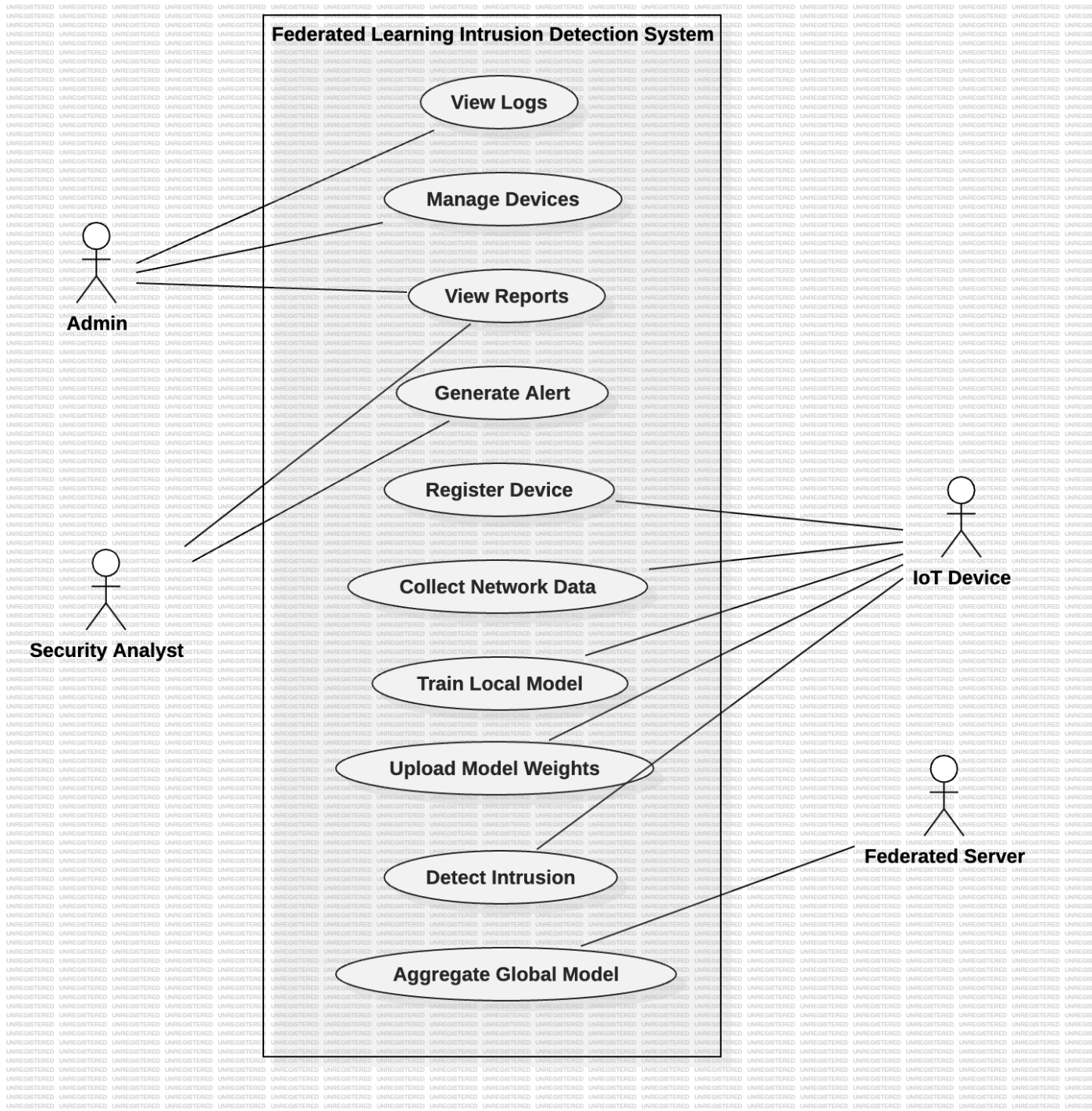
6.1 Class Diagram



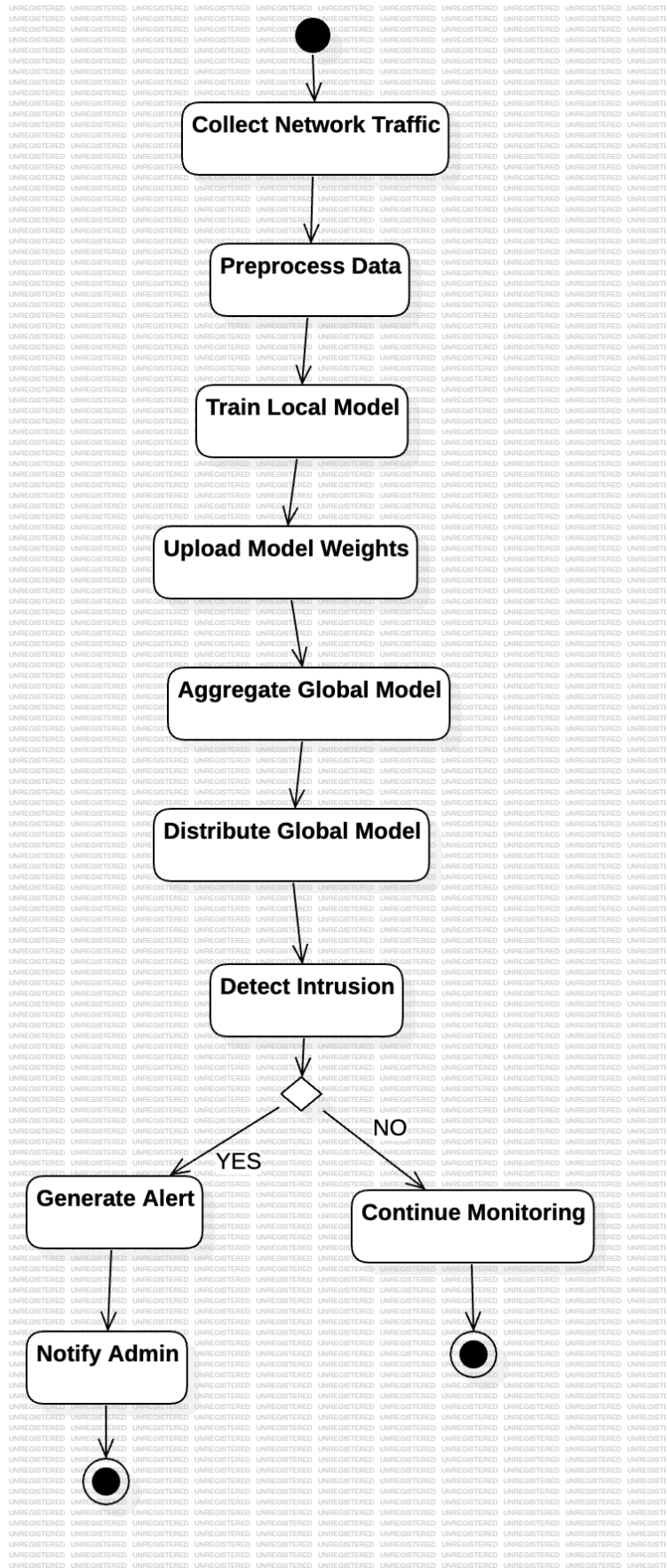
6.2 Sequence Diagram



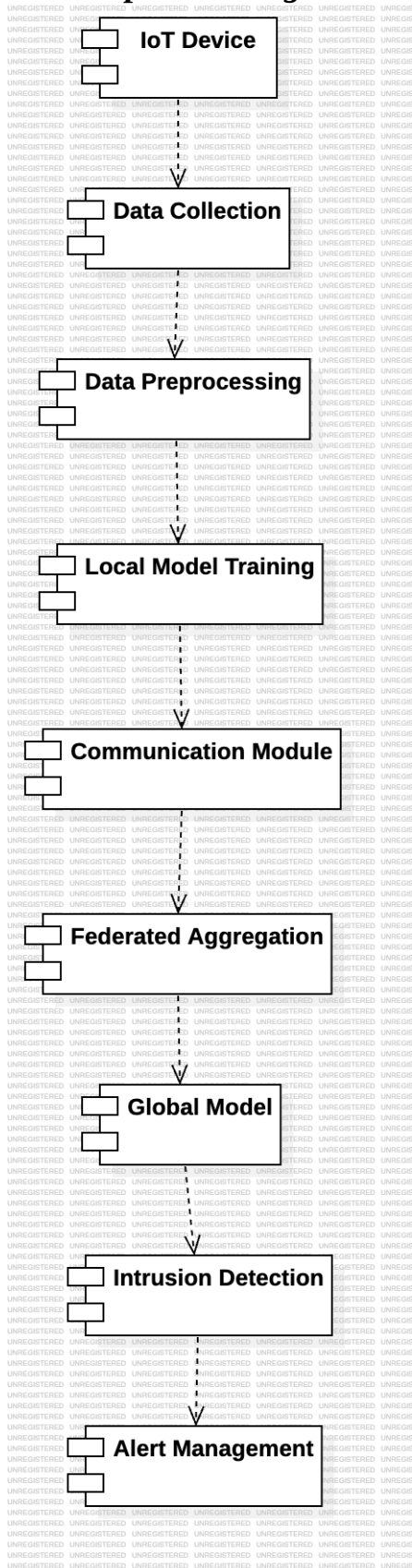
6.3 Use Case Diagram



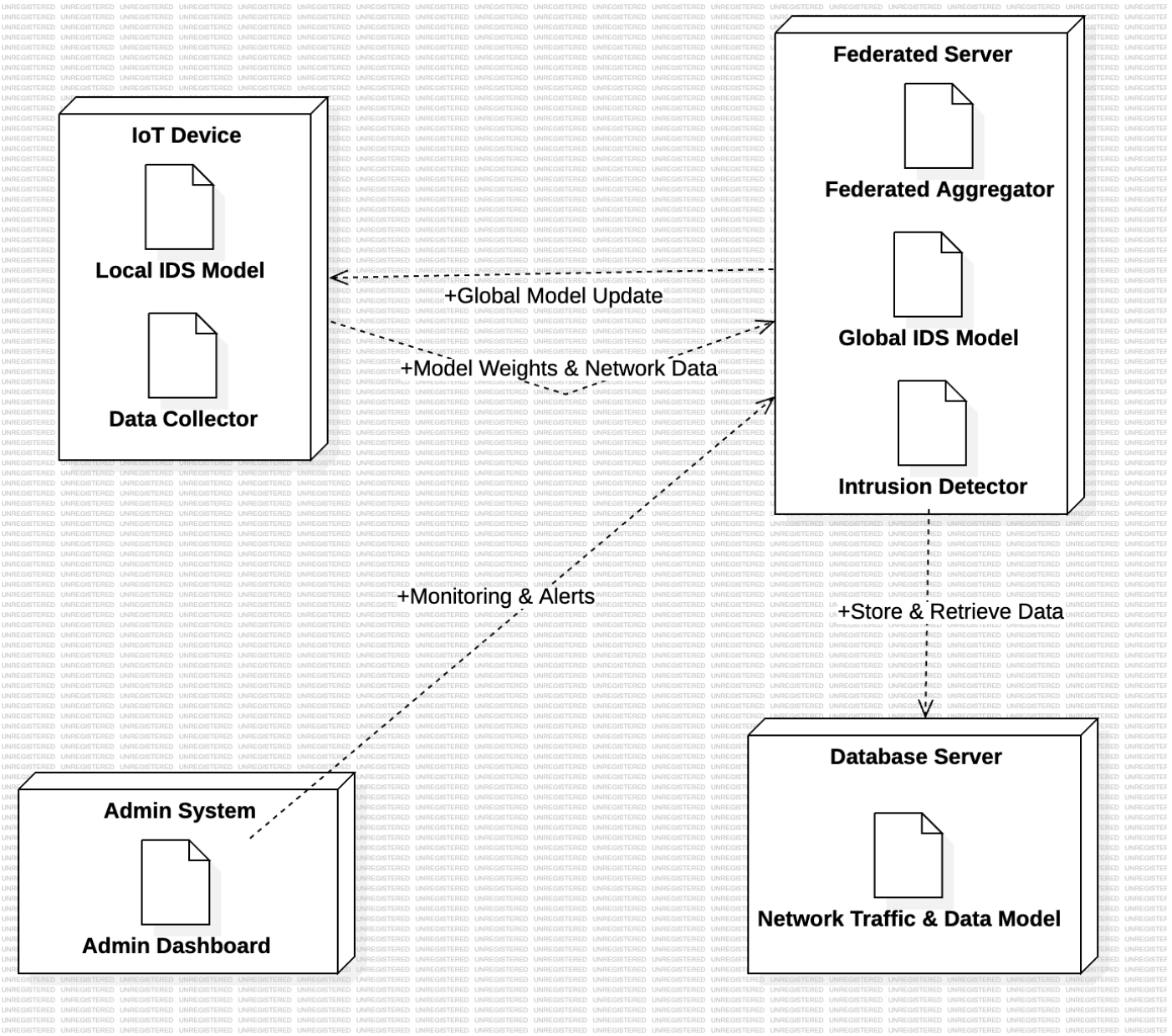
6.4 Activity Diagram



6.5 Component Diagram



6.6 Deployment Diagram



Appendix A: Glossary

The following terms are used throughout this document.

Term	Description
IoT	<i>Internet of Things – A network of interconnected smart devices that communicate over the internet.</i>
IDS	<i>Intrusion Detection System – A security system that monitors network traffic and detects malicious activities.</i>
FL	<i>Federated Learning – A distributed machine learning approach where multiple clients collaboratively train a model without sharing their raw data.</i>
TCN	<i>Temporal Convolutional Network – A deep learning architecture designed for sequential and time-series data using causal and dilated convolutions.</i>
FedAvg	<i>Federated Averaging – An algorithm used to aggregate locally trained model parameters into a global model.</i>
Client	<i>An IoT device or local node that performs model training using its own dataset.</i>
Global Model	<i>The model generated by aggregating updates received from all participating clients.</i>
Local Model	<i>The machine learning model trained independently on a client's local dataset.</i>
Communication Round	<i>One complete cycle of local model training, parameter transmission, aggregation, and global model distribution.</i>
Dataset	<i>A collection of labeled network traffic records used for training and testing the intrusion detection model.</i>
Attack Classification	<i>The process of identifying whether network traffic is normal or belongs to a specific attack category.</i>
ROC-AUC	<i>Receiver Operating Characteristic – Area Under Curve, used to evaluate classification performance.</i>
Precision	<i>The proportion of correctly predicted attacks among all predicted attacks.</i>
Recall	<i>The proportion of actual attacks correctly identified by the model.</i>
F1-Score	<i>The harmonic mean of Precision and Recall.</i>

Appendix B: Abbreviations

<i>Abbreviation</i>	<i>Full Form</i>
<i>AI</i>	<i>Artificial Intelligence</i>
<i>CNN</i>	<i>Convolutional Neural Network</i>
<i>CPU</i>	<i>Central Processing Unit</i>
<i>DoS</i>	<i>Denial of Service</i>
<i>DDoS</i>	<i>Distributed Denial of Service</i>
<i>FL</i>	<i>Federated Learning</i>
<i>GPU</i>	<i>Graphics Processing Unit</i>
<i>IDS</i>	<i>Intrusion Detection System</i>
<i>IoT</i>	<i>Internet of Things</i>
<i>ML</i>	<i>Machine Learning</i>
<i>RAM</i>	<i>Random Access Memory</i>
<i>ROC</i>	<i>Receiver Operating Characteristic</i>
<i>TCN</i>	<i>Temporal Convolutional Network</i>
<i>TCP</i>	<i>Transmission Control Protocol</i>
<i>UDP</i>	<i>User Datagram Protocol</i>

Appendix C: Future Scope

Although the proposed system provides an efficient and privacy-preserving intrusion detection framework, there is scope for further improvement. Future versions of the software can support real-time deployment on physical IoT devices instead of simulated federated clients. Advanced federated optimization algorithms can be incorporated to improve model convergence and reduce communication costs.

The system can also be extended to include explainable artificial intelligence (XAI) techniques to provide better insight into intrusion detection decisions. Support for additional IoT datasets and emerging attack categories can further improve the robustness of the model. Integration with cloud-edge computing platforms and blockchain-based secure model aggregation may enhance scalability and security in large-scale IoT environments.

Appendix D: Conclusion

The proposed software presents a lightweight and privacy-preserving intrusion detection system for resource-constrained IoT devices using Temporal Convolutional Networks and Federated Learning. By allowing IoT devices to train models locally and share only model parameters, the system significantly reduces privacy risks and communication overhead while maintaining effective intrusion detection performance.

The use of the TCN model enables efficient learning of temporal patterns in network traffic, making the solution suitable for devices with limited computational resources. The modular architecture of the system supports scalability, maintainability, and future enhancements. Overall, the proposed software provides an effective framework for improving the security of IoT networks while preserving user data privacy.

Bibliography

1. Anbazhagan, S., Kavitha, M., Murugan, S., Kumar, K. M., Rashid, M. M., & Hilal, A. M. Federated Learning-Based Lightweight Intrusion Detection for Resource-Constrained IoT Devices Using Temporal Convolutional Networks. Scientific Reports, Nature Portfolio, 2025. (Primary reference paper)
2. IEEE Recommended Practice for Software Requirements Specifications (IEEE 830).
3. TensorFlow Documentation.
4. Keras Documentation.
5. Scikit-learn Documentation.
6. NumPy Documentation.
7. Pandas Documentation.